MAE162
CAPSTONE I/II

Lab 1: ROS2 Introduction and System Integration

Version 0.1.0

162 TAs and Maclab members

March 31, 2026

Warning!

- No food and drinks are allowed in the lab.
- Be careful with the hardware especially the electronics. Any liquid spill can cause a short circuit, potentially significantly delaying your progress.
- Never generate a short circuit.
- Connect electric wires only when you are sure about what you are doing.
- Be gentle to the buttons! You are liable to the damage made to the robot and its electronics.
- Wash your hands after each experiment, some components may contain some lead.
- Do not run experiments on the table for the chance the robot may fall on the floor and get damaged.

Contents

1 System Architecture **3**

2 ROS2 Introduction **6**

2.1 Core Concepts 6

2.2 Docker for ROS2 7

3 Lab Activities **8**

3.1 Task 1: Set Up Your Repository 8

3.2 Task 2: Flash the Firmware 10

3.3 Task 3: Bring Up the ROS2 Environment and NUEVO UI 11

3.4 Task 4: DC Motor PI Tuning and Step Response 13

4 Lab Deliverables **15**

1 System Architecture

Figure 1 shows the full system architecture of Project NUEVO. The system is divided into several blocks that communicate over well-defined interfaces. Understanding this structure is essential for debugging and extending the robot. For a comprehensive reference covering all subsystems, see `docs/manual/MANUAL.md`.

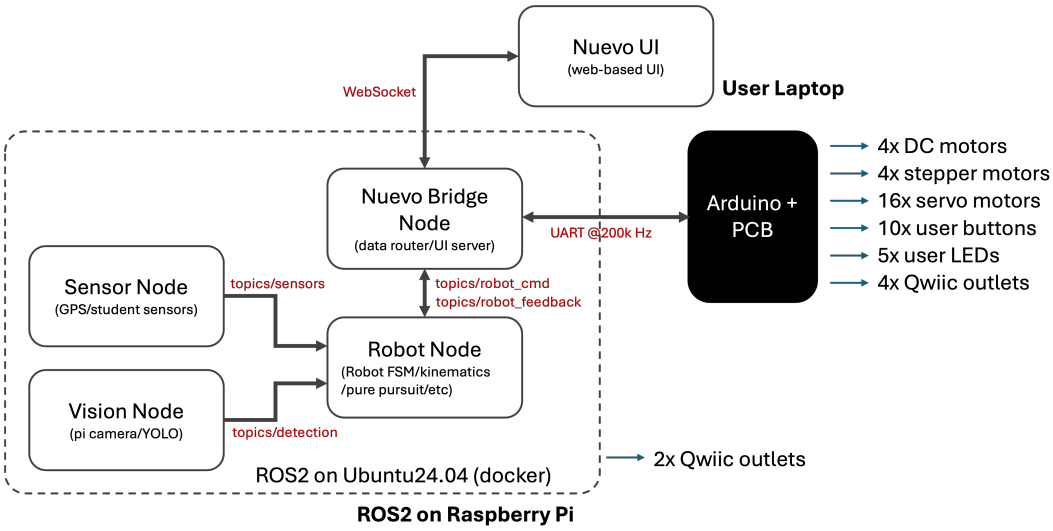

Figure 1: System Architecture.

PCB and Firmware

The custom PCB hosts an **Arduino Mega 2560** microcontroller and breaks out the following hardware channels:

- **4 DC motor channels** with quadrature encoder feedback
- **4 stepper motor channels** (A4988/DRV8825 drivers)
- **16 servo channels** via a PCA9685 I^2C PWM controller
- **IMU** (ICM-20948: accelerometer, gyroscope, magnetometer)
- **Voltage monitoring** for the battery rail, 5 V rail, and servo rail
- **User I/O**: buttons, discrete LEDs, and NeoPixel RGB LEDs
- **Limit switches** and expandable ultrasonic sensor slots

The firmware runs hard real-time control loops: DC motor velocity control at 800 Hz, stepper pulse generation at 10 kHz, and a soft scheduling loop at 100–200 Hz for UART communication and sensor reads. The firmware is the only component that directly controls the actuators and reads hardware encoders; all other layers communicate with it through the bridge.

For full firmware details refer to:

- `firmware/README.md` — build, upload, and configuration overview
- `firmware/arduino/src/config.h` — all compile-time settings (channel counts, gains, rates)
- `firmware/docs/architecture.md` — runtime task structure and startup order
- `docs/COMMUNICATION_PROTOCOL.md` — TLV protocol, message catalog, and state machine
- `tlv_protocol/TLV_Payloads.md` — exact payload byte layouts and IDs

Bridge

The bridge is a Python process running on the **Raspberry Pi 5**. It owns the single UART connection to the Arduino (`/dev/ttyAMA0` at 200 kbaud) and translates between the compact TLV wire protocol and higher-level interfaces. In addition to the transport layer, the bridge runs a **FastAPI/WebSocket server** that serves the NUEVO web UI. When running inside the ROS2 Docker container, the bridge also publishes and subscribes to ROS2 topics and services, acting as the boundary between the real-time firmware world and the robot application layer.

Refer to:

- `ros2_ws/BRIDGE_RUNTIME.md` — bridge architecture, shared runtime model, and reset recovery
- `docs/COMMUNICATION_PROTOCOL.md` — TLV protocol, message catalog, and state machine

NUEVO UI

The NUEVO UI is a **React** web application served by the bridge backend. It provides a dashboard for testing and monitoring all hardware in real time: motor speed setpoints and feedback, stepper positions, servo angles, LED color, user button states, IMU readings, voltage levels, and odometry. You access it from any browser on the same network by navigating to the Raspberry Pi's IP address at port 8000.

Refer to:

- `ros2_ws/README.md` — build instructions for the frontend and how to start the bridge that serves the UI

Robot Node

The `robot` ROS2 package is where high-level **decision-making and path planning** code lives. It subscribes to sensor and state topics published by the bridge, computes motion commands, and publishes velocity setpoints back to the bridge for execution. In this lab the robot node is a scaffold; you will extend it in future labs.

Sensor Node

The `sensors` ROS2 package handles **Raspberry Pi–connected sensors** that are not part of the Arduino firmware, such as GPS receivers and external range sensors (LiDAR, ultrasonic modules, etc.). It publishes sensor data as ROS2 topics for the robot node to consume.

Vision Node

The `vision` ROS2 package handles the **camera pipeline and object detection**. It reads frames from the camera, runs inference (e.g. YOLO), and publishes detection results as ROS2 topics. The vision dependencies (OpenCV, libcamera, YOLO) are heavier than the rest of the stack and are added to the Docker image separately.

Refer to the following for the Robot, Sensor, and Vision nodes:

- `ros2_ws/ROS2_NODES_DESIGN.md` — package layout, design principles, and topic/service naming
- `ros2_ws/README.md` — Docker workflow, node startup order, and ROS2 API overview

Why ROS2 and Docker?

With this architecture, each component has a single responsibility and communicates through a standardised message-passing system. Adding a new sensor, planner, or perception algorithm means writing a new node that subscribes and publishes standard topics — the rest of the system does not change. Docker ensures that every team member and every Raspberry Pi runs the *exact same* software environment regardless of the host operating system, eliminating "it works on my machine" problems and making deployment to the robot a one-command step.

2 ROS2 Introduction

2.1 Core Concepts

ROS2 (Robot Operating System 2) is an open-source middleware framework for building robot software. Despite the name it is not an operating system: it is a set of libraries, tools, and conventions that run on top of a standard OS and let independently developed programs communicate with each other cleanly.

Nodes are the basic executable units. Each node is a process (or a thread within a process) that performs one focused task — reading a sensor, running a controller, publishing odometry. In this project: `bridge`, `robot`, `sensor_node`, and `vision_node` are each a separate node.

Topics are named communication channels that carry a stream of messages. A node that produces data *publishes* to a topic; a node that needs that data *subscribes* to it. Topics are asynchronous and many-to-many. For example, the bridge node publishes DC motor state on `/dc_state_all` and the robot node subscribes to that topic.

Messages define the data structure for a topic. They are declared in `.msg`

files inside the `bridge_interfaces` package and compiled into language-specific code automatically. `bridge_interfaces/msg/DCStateAll` and `bridge_interfaces/msg/SystemPower` are examples used in this project.

Services are synchronous request–reply interactions. A node that handles a service advertises it; another node calls it and waits for a response. Use services for discrete operations where you need to know whether the action succeeded. In this project, firmware state transitions (IDLE → RUNNING, triggering an E-stop) use the `/set_firmware_state` service.

Packages are the unit of organisation and distribution. A package groups related nodes, message definitions, and launch files. This project has five packages: `bridge_interfaces` (message/service definitions), `bridge` (ROS wrapper around the firmware bridge), `robot` (motion planning), `sensors` (Pi-side sensors), and `vision` (camera and detection).

Keywords for further learning: ROS2 publisher/subscriber, colcon build, `ros2 topic echo`, `ros2 node info`, QoS policies, launch files, parameter server, tf2 transforms, RViz2, rqt.

2.2 Docker for ROS2

Modern robots are developed by teams working on different machines with different operating systems — macOS, Windows, Ubuntu. ROS2 Jazzy officially targets Ubuntu 24.04; running it natively on macOS or Windows requires significant manual setup that is fragile and hard to reproduce. Docker solves this by packaging the entire ROS2 environment — OS libraries, ROS installation, workspace build — into a single *container image*. Everyone on the team starts the same image, so the software environment is *identical* whether you are developing on a laptop or deploying to the Raspberry Pi. This also means updating the environment is a `docker compose build` rather than a multi-step manual procedure.

In this project the Docker image contains ROS2 Jazzy, the project workspace, and all Python runtime dependencies. Two compose files are provided:

- `ros2_ws/docker/docker-compose.rpi.yml` — for the Raspberry Pi with real hardware
- `ros2_ws/docker/docker-compose.vm.yml` — for macOS / Windows development (mock bridge)

Keywords for further learning: Dockerfile, Docker image vs. container, `docker compose`, volume mounts, container networking, `docker exec`, base images.

3 Lab Activities

3.1 Task 1: Set Up Your Repository

In this course you will modify the Project NUEVO codebase to add features and tune your robot. Each team should work in their own copy of the repository so that your changes do not affect other teams and so you can safely experiment.

1.1 Fork the Repository

1. Go to github.com/pochihh/Project-NUEVO.
2. Click **Fork** (top-right corner) to create a copy under your own GitHub account. *(keyword: fork)*
3. On your forked repository page, go to **Settings** → **Collaborators** and add your teammates as collaborators so everyone can push to the same fork. *(keyword: collaborators, repository permissions)*

1.2 Clone and Collaborate with Git

Git lets your whole team work on the same codebase simultaneously. The basic workflow is:

1. **Clone** your fork to your local machine. This downloads the full repository history.

```
git clone https://github.com/<your-username>/Project-NUEVO.git
cd Project-NUEVO
```

(keyword: clone, remote)

2. **Make changes**, then **stage** the files you want to save.

```
git add <filename>
```

(keyword: staging area, working tree)

3. **Commit** to record a snapshot with a short message describing what you did.

```
git commit -m "brief description of change"
```

(keyword: commit, commit message)

4. **Push** to upload your commits to GitHub so teammates can see them.

```
git push
```

(keyword: push, origin)

5. **Pull** to download your teammates' latest commits before you start working.

```
git pull
```

(keyword: pull, merge, rebase, merge conflict)

1.3 Pull Updates from the Original Repository

Bug fixes and new features will be released to the original repository during the course. To bring them into your fork, add the original repository as a second remote and pull from it:

1. Add the original repository as the `upstream` remote (one-time setup):

```
git remote add upstream https://github.com/pochihh/Project-NUEVO.git
```

(keyword: remote, upstream)

2. Fetch and merge the latest changes from upstream into your local branch:

```
git fetch upstream
git merge upstream/main
```

(keyword: fetch, merge, fast-forward)

3. Push the result to your fork:

```
git push
```

3.2 Task 2: Flash the Firmware

The Arduino Mega firmware must be flashed onto the robot before the bridge can communicate with it. Full details and troubleshooting are in `firmware/README.md`.

2.1 Prerequisites

1. Install **Arduino IDE 2.x** and the **Arduino AVR Boards** core.
2. Increase the serial buffer size so the firmware UART link does not overflow. Create or edit `platform.local.txt` for the installed AVR core and add:

```
compiler.cpp.extra_flags=-DSERIAL_RX_BUFFER_SIZE=512
↪  -DSERIAL_TX_BUFFER_SIZE=256
```

   Typical file locations (replace `1.8.7` with your installed AVR core version):

   macOS:

```
~/Library/Arduino15/packages/arduino/hardware/avr/1.8.7/platform.local.
↪  txt
```

   Windows:

```
C:\Users\<your-user>\AppData\Local\Arduino15\packages\arduino\hardware\
↪  avr\1.8.7\platform.local.txt
```

2.2 Build and Upload

1. Open `firmware/arduino/arduino.ino` in Arduino IDE.
2. Select board: **Arduino Mega or Mega 2560**.
3. Select the correct COM / serial port for the connected Arduino.
4. Click **Verify** to confirm the firmware compiles without errors.
5. Click **Upload** to flash the firmware onto the board.

After a successful upload the board resets and the firmware starts running. Confirm by opening the Arduino IDE Serial Monitor at 115 200 baud and checking for heartbeat output.

3.3 Task 3: Bring Up the ROS2 Environment and NUEVO UI

In this task you will start the Docker container on the Raspberry Pi, launch the bridge node, and verify all hardware using the NUEVO web UI. Full setup details are in `ros2_ws/README.md`.

3.1 Raspberry Pi UART Setup (one-time)

The bridge communicates with the Arduino over the RPi hardware UART. Enable it by editing the RPi boot configuration files:

1. Add the following lines to `/boot/firmware/config.txt`:

```
enable_uart=1
dtoverlay=uart0-pi5
```

2. Remove any `console=serial0` or `console=ttyAMA0` entries from `/boot/firmware/cmdline.txt` so the OS does not claim the port.

3. Reboot and verify: `ls -l /dev/ttyAMA0` should show a character device owned by the `dialout` group.

3.2 Build the Frontend (one-time)

The bridge serves the NUEVO web UI as a pre-built static bundle. Build it once from the repository root:

```
cd nuevo_ui/frontend
npm install
npm run build
cd ../..
cp -r nuevo_ui/frontend/dist/. nuevo_ui/backend/static/
```

3.3 Start the Docker Container

From the repository root on the Raspberry Pi:

```
COMPOSE=ros2_ws/docker/docker-compose.rpi.yml
docker compose -f $COMPOSE build
docker compose -f $COMPOSE up -d
docker compose -f $COMPOSE logs -f ros2_runtime
```

Wait until the log shows that `colcon build` finished and the container is ready.

3.4 Launch the Bridge Node

Open a shell inside the running container and start the bridge:

```
docker compose -f $COMPOSE exec ros2_runtime bash
source /opt/ros/jazzy/setup.bash
source /ros2_ws/install/setup.bash
ros2 run bridge bridge
```

The bridge node connects to the Arduino, starts the WebSocket server, and begins publishing ROS2 topics.

3.5 Access the NUEVO UI

Open a browser on any device on the same network and navigate to:

```
http://<raspberry-pi-ip>:8000
```

Login credentials: username `user`, password `162`. **Change your password after first login.**

Use the UI to test each hardware feature:

- Set stepper motor positions and verify movement.
- Command servo angles and observe the mechanical response.
- Change LED colors via the colour picker.
- Press each user button and confirm the indicator updates.
- Check IMU readings (accelerometer, gyroscope) for expected values.
- Monitor the battery voltage level on the power panel.

3.4 Task 4: DC Motor PI Tuning and Step Response

In this task you will tune the PI velocity controller for one DC motor and record step responses to evaluate your design. The goal is to transition from the default gains to a set of gains that you can justify from a simple model of the motor dynamics.

4.1 Model-Based PI Design (Optional)

A DC motor's velocity dynamics can be approximated as a first-order linear system:

$$G(s) = \frac{K}{\tau s + 1} \tag{1}$$

where K is the steady-state gain (ticks/s per PWM unit) and τ is the time constant (seconds). You can identify both parameters from an open-loop step response.

Step 1: Measure the open-loop step response. In the NUEVO UI, switch the motor to **PWM mode** and apply a step change in PWM from 0 to a fixed value (e.g. 50 %). Record the velocity $\omega(t)$ response using the UI's plot feature.

From the recorded curve:

- $K = \omega_{ss}/u$ (steady-state velocity divided by the applied PWM)
- τ = the time at which $\omega(t)$ first reaches 63.2 % of ω_{ss}

Step 2: Choose the closed-loop bandwidth. A PI controller with plant $G(s)$ gives a second-order closed-loop response. Matching coefficients to the standard form $s^2 + 2\zeta\omega_n s + \omega_n^2$:

$$K_p = \frac{2\zeta\omega_n\tau - 1}{K}, \qquad K_i = \frac{\omega_n^2\,\tau}{K} \tag{2}$$

Choose a desired damping ratio $\zeta = 0.7$ (a good default that balances speed and overshoot). Select ω_n based on a rise-time or settling-time specification of your choice, then compute K_p and K_i from Equations (2).

4.2 Simple Observation (Alternative to 4.1)

If you prefer a faster approach, skip the model identification and instead adjust K_p and K_i manually while observing the step response: increase K_p to speed up the response, increase K_i to eliminate steady-state error, and reduce both if the response oscillates.

4.3 Record and Compare Step Responses

Using the NUEVO UI, command a velocity step from 0 to 1000 ticks/s and record the response for:

1. The **default** PI gains (as shipped in `firmware/arduino/src/config.h`).
2. Your **tuned** gains from Task 4.1 or 4.2.

On each plot, annotate:

- Rise time t_r (time to go from 10 % to 90 % of the setpoint)
- Percent overshoot M_p

- Settling time t_s (time to stay within $\pm 5\,\%$ of the setpoint)
- Steady-state error e_{ss}

A MATLAB script (`plot_step_response.m`) is provided in the lab repository to load the exported CSV, compute all four metrics automatically, and produce an annotated plot. Figure 2 shows an example output.

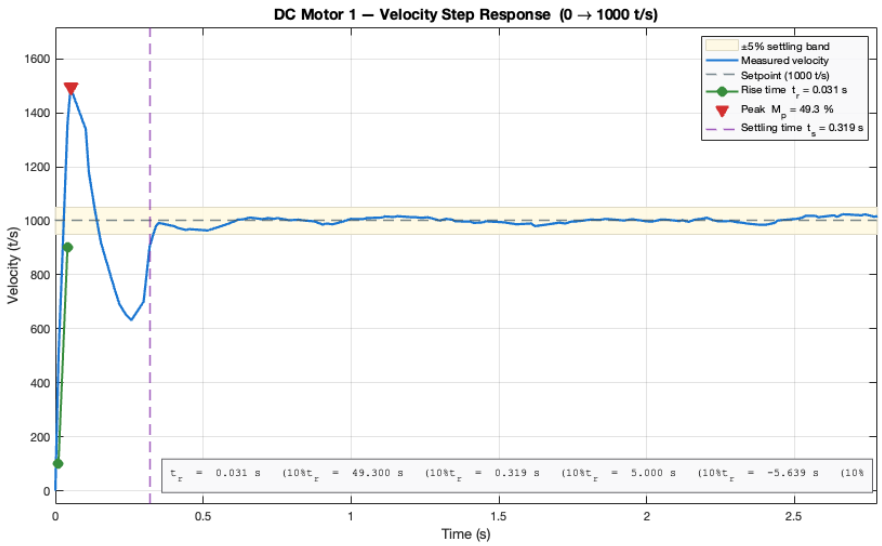

Figure 2: Example annotated velocity step response ($0 \rightarrow 1000$ ticks/s). The shaded region is the $\pm 5\,\%$ settling band.

4 Lab Deliverables

Show your final step response plots to the TAs. Your plots should include both the default and tuned step responses with the performance metrics annotated. Justify your tuned gains: explain how you arrived at your K_p and K_i values, whether through the model-based approach or manual tuning, and discuss the trade-offs you observed between rise time, overshoot, and settling time.